Вернуться к 'Рекурсия. Функции высших порядков'

Тест начат	Пятница, 25 Сентябрь 2026, 23:26
Состояние	Завершенные
Завершен	Суббота, 26 Сентябрь 2026, 00:23
Прошло времени	57 мин. 44 сек.
Баллы	15,00/15,00
Оценка	**5,00** из 5,00 (**100**%)

Вопрос **1**

Верно

Баллов: 1,00 из 1,00

Выберите все варианты, которые реализуют функцию any

```
any :: (a -> Bool) -> [a] -> Bool
```

где списки являются конечными, правильно определенными списками и где предикат p всегда возвращает либо True либо False

Выберите один или несколько ответов:

- [x] `any p = not . null . dropWhile (not . p)` ✔
- [ ] `any p = map p . or`
- [x] `any p xs = length (filter p xs) > 0` ✔
- [x] `any p = or . map p` ✔
- [ ] `any p = null . filter p`
- [ ] `any p xs = foldr (||) True (map p xs)`
- [x] `any p xs = not (all (\ x -> not (p x)) xs)` ✔

Вопрос **2**

Верно

Баллов: 1,00 из 1,00

Выберите все правильные определения оператора возведения в степень ^ для неотрицательных целых чисел (в том числе 0).

Выберите один или несколько ответов:

- [ ]
```
m ^ 0 = 0
m ^ n = m * m ^ (n - 1)
```
- [ ]
```
m ^ 1 = m
m ^ n = m * m ^ (n - 1)
```
- [x]
```
m ^ 0 = 1
m ^ n = m * (^) m (n - 1)
```
- [ ]
```
m ^ 0 = 1
m ^ n = m * m * m ^ (n - 2)
```
- [ ]
```
m ^ 0 = 1
m ^ n = m * m ^ n - 1
```
- [ ]
```
m ^ 0 = 1
m ^ n = n * n ^ (m - 1)
```
- [ ]
```
m ^ 0 = 1
m ^ n = (m * m) ^ (n - 1)
```
- [x]
```
m ^ 0 = 1
m ^ n = m * m ^ (n - 1)
```

Вопрос **3**

Верно

Баллов: 1,00 из 1,00

Выражение `filter even (map (+1) [1..5])` равно

Выберите один ответ:

- ( ) `[2,4]`
- (•) `[2,4,6]` ✔
- ( ) `[]`
- ( ) `[1,3,5]`
- ( ) `[3,5]`

Вопрос **4**

Верно

Баллов: 1,00 из 1,00

Используя следующее определение, покажите как вычисляется `init [1, 2, 3]`

```
init :: [a] -> [a]
init [_] = []
init (x : xs) = x : init xs
```

Выберите один ответ:

a.
```
init [1,2,3]
= { applying init }
1 : init [2,3]
= { applying init }
1 : 2 : init [3]
= { applying init }
1 : 2 : []
= { list notation }
[1,2]
```
✔

b.
```
init [1,2,3]
= { applying init }
1 : init [2,3]
= { applying init }
1 : []
= { list notation }
[1]
```

c.
```
init [1,2,3]
= { applying init }
[1,2] : init [3]
= { applying init }
[1,2] : []
= { list notation }
[1,2]
```

d.
```
init [1,2,3]
= { applying init }
1 : init [2,3]
= { applying init }
1 : 2 : init [3]
= { applying init }
1 : 2 : 3 : init []
= { applying init }
1 : 2 : 3 : []
= { list notation }
[1,2,3]
```

Вопрос **5**

Верно

Баллов: 1,00 из 1,00

Выражение `zip [1, 2] ['a', 'b', 'c']` равно

Выберите один ответ:

- `[(1,'a'),(2,'b'),(2,'c')]`
- `An error`
- `[(1,'a'),(2,'b')]` ✔
- `[(1,'a'),(2,'b'),(3,'c')]`
- `([1,2],['a','b','c'])`

Вопрос **6**

Верно

Баллов: 1,00 из 1,00

Выберите все варианты, которые реализуют функцию all

```
all :: (a -> Bool) -> [a] -> Bool
```

где списки являются конечными, правильно определенными списками и где предикат p всегда возвращает либо True либо False

Выберите один или несколько ответов:

- [x] `all p xs = and (map p xs)` ✔
- [ ] `all p = map p . and`
- [ ] `all p xs = map p (and xs)`
- [x] `all p = foldr (&&) True . map p` ✔
- [x] `all p = not . any (not . p)` ✔
- [x] `all p = and . map p` ✔
- [x] `all p xs = foldl (&&) True (map p xs)` ✔
- [ ] `all p xs = foldr (&&) False (map p xs)`

Вопрос **7**

Верно

Баллов: 1,00 из 1,00

Выберите правильное определение для функции, которая выбирает n-й элемент списка. Отсчет с 0.

```
(!!) :: [a] -> Int -> a
```

Выберите один ответ:

a.
```
(x : _) !! 0 = [x]
(_ : xs) !! n = xs !! (n - 1)
```

b.
```
(x : _) !! 1 = x
(_ : xs) !! n = xs !! (n - 1)
```

c.
```
(x : _) !! 0 = x
(_ : xs) !! n = xs !! (n - 1)
[] !! n = 0
```

d. ✔
```
(x : _) !! 0 = x
(_ : xs) !! n = xs !! (n - 1)
```

Вопрос **8**

Верно

Баллов: 1,00 из 1,00

Выберите правильное определение для функции, которая объединяет список списков:

```
concat :: [[a]] -> [a]
```

Выберите один ответ:

```
concat [] = []
concat (xs : xss) = xs : concat xss
```

```
concat [[]] = []
concat (xs : xss) = xs ++ concat xss
```

✔
```
concat [] = []
concat (xs : xss) = xs ++ concat xss
```

```
concat [] = [[]]
concat (xs : xss) = xs ++ concat xss
```

Вопрос **9**

Верно

Баллов: 1,00 из 1,00

Выберите правильное определение для функции, которая определяет, является ли указанное значение элементом списка:

```
elem :: Eq a => a -> [a] -> Bool
```

Выберите один ответ:

```
elem _ [] = False
elem x (y : ys)
  | x == y = True
  | otherwise = elem x ys
```

```
elem _ [] = True
elem x (y : ys)
  | x == y = True
  | otherwise = elem x ys
```

```
elem _ [] = False
elem x (y : ys)
  | x == y = True
  | otherwise = elem x (y : ys)
```

```
elem _ [] = False
elem x (y : ys) = x == y
```

Вопрос **10**

Верно

Баллов: 1,00 из 1,00

Выберите правильное определение для функции, которая выдает список с n одинаковыми элементами:

```
replicate :: Int -> a -> [a]
```

Выберите один ответ:

```
replicate 0 _ = []
replicate n x = replicate (n - 1) x : x
```

```
replicate 0 _ = []
replicate n x = x : replicate (n - 1) x
```

```
replicate 1 _ = []
replicate n x = replicate (n - 1) x ++ [x]
```

```
replicate 1 x = x
replicate n x = x : replicate (n - 1) x
```

Вопрос **11**

Верно

Баллов: 1,00 из 1,00

Выберите все варианты, которые реализуют функцию dropWhile

```
dropWhile :: (a -> Bool) -> [a] -> [a]
```

где списки являются конечными, правильно определенными списками и где предикат p всегда возвращает либо True либо False

Выберите один ответ:

-
```
dropWhile _ [] = []
dropWhile p (x : xs)
  | p x = dropWhile p xs
  | otherwise = x : xs
```
✔

-
```
dropWhile p = foldr (\ x acc -> if p x then acc else x : acc)
```

-
```
dropWhile _ [] = []
dropWhile p (x : xs)
  | p x = dropWhile p xs
  | otherwise = xs
```

-
```
dropWhile p = foldl add []
  where add [] x = if p x then [] else [x]
        add acc x = x : acc
```

Вопрос **12**

Верно

Баллов: 1,00 из 1,00

Выберите эквивалентое определение списка `[f x | x <- xs, p x]` через функции высших порядков

Выберите один ответ:

-
```
map f (filter p xs)
```
✔

-
```
map f (takeWhile p xs)
```

-
```
filter p (map f xs)
```

-
```
map p (map f xs)
```

Вопрос **13**

Верно

Баллов: 1,00 из 1,00

Используя следующее определение, показать, как вычисляется

`length [1, 2, 3]`

```
length :: [a] -> Int
length [] = 0
length (_ : xs) = 1 + length xs
```

Выберите один ответ:

a.

```
length [1,2,3]
= { applying length }
1 + length [2,3]
= { applying length }
1 + (1 + length [3])
= { applying length }
1 + (1 + (1 + length []))
= { applying length }
1 + (1 + (1 + 1))
= { applying + }
1 + (1 + 2)
= { applying + }
1 + 3
= { applying + }
4
```

b.

```
length [1,2,3]
= { applying length }
1 + length [2,3]
= { applying length }
1 + (1 + length [3])
= { applying length }
1 + (1 + 1)
= { applying + }
3
```

c.

```
length [1,2,3]
= { applying length }
1 + length [2,3]
= { applying length }
1 + (2 + length [3])
= { applying length }
1 + (2 + (3 + length []))
= { applying length }
1 + (2 + (3 + 0))
= { applying + }
6
```

d.

```
length [1,2,3]
= { applying length }
1 + length [2,3]
= { applying length }
1 + (1 + length [3])
= { applying length }
1 + (1 + (1 + length []))
= { applying length }
1 + (1 + (1 + 0))
= { applying + }
1 + (1 + 1)
= { applying + }
1 + 2
= { applying + }
3
```

✔

Вопрос **14**

Верно

Баллов: 1,00 из 1,00

Выберите все варианты, которые реализуют функцию map

```
map :: (a -> b) -> [a] -> [b]
```

где списки являются конечными, правильно определенными списками и где предикат p всегда возвращает либо True либо False

Выберите один ответ:

- `map f = foldr (\ x xs -> f x ++ xs) []`
- `map f = foldl (\ xs x -> xs ++ [f x]) []` ✔
- `map f = foldr (\ x xs -> xs ++ [f x]) []`
- `map f = foldl (\ xs x -> f x : xs) []`

Вопрос **15**

Верно

Баллов: 1,00 из 1,00

Какие из предложенных выражений эквивалентны [1, 2, 3, 4]:

Выберите один ответ:

- `map (<= 4) [1..]`
- `1:2:3:4`
- `1++2++3++4`
- `take 4 (iterate (+1) 1)` ✔
- `filter (<= 4) (repeat 1)`